Recursion, Recurrences, and Divide-and-Conquer — Exercises

A02-E01 Complete stable MERGE

Let $L = [0_a, 3_b, 7_c]$ and $R = [1_d, 3_e, 6_f]$, where subscripts label original records. List the indices before each key comparison, the selected record, and the output prefix; include the cleanup after exhaustion. Count key comparisons, copies into temporary buffers, and writes back. If equal keys favor the right side, is the output still a sort? Is it stable?

A02-E02 Sentinels and aliases have assumptions

Legal keys include a maximum M. A merge uses M as a sentinel and emits exactly m records, favoring the left side on equality. Trace $L = [M], R = [M, M]$ to identify an out-of-bounds read. Separately, let both inputs be views of the two halves of $A = [2, 4, 1, 3]$, with output written into A. Explain how the first overwrite corrupts the next read. Repair both problems without restricting the legal key domain.

A02-E03 Odd lengths and peak space

Apply the chapter's MERGE-SORT to $[6_a, 2_b, 5_c, 1_d, 4_e]$. List every internal call (p, q, r) and its merged result. Count cumulative buffer copies, output writes, and the largest single buffer demand. Assuming each MERGE releases its buffers before returning, distinguish cumulative allocation, peak buffer space, and live call frames.

A02-E04 Exact recurrence solutions

For $n = 2^h$, solve $U(1) = 3, U(n) = 2U(n/2) + 5n$, and $W(1) = 0, W(n) = 2W(n/2) + n - 1$ exactly by expansion or induction. Evaluate both at $n = 8$. Why is the worst-case $m - 1$ merge count used in W not the actual comparison count for every input?

A02-E05 Θ bounds beyond powers of two

Without the master theorem, prove that this MERGE-SORT takes $\Theta(n \lg n)$ time for every $n \geq 2$. Give actual interval sizes, per-level upper and lower bounds, stopping depth, and total base-case work. Do not round n to a power of two and assume runtime monotonicity without proof.

A02-E06 Model changes and an early-exit variant

On sorted input, compare insertion sort and this standard merge sort. After sorting both halves, a variant tests $A[q] \le A[q+1]$ and skips MERGE when true. Prove correctness and stability, and analyze sorted-input time. How must the cost model change if a key comparison can read $\Theta(L)$ characters?

A02-E07 Maximum-subarray states and interval

Let E_i be the best nonempty contiguous sum ending at i, and B_i the best nonempty sum among the first i records. For $A = [5, -9, 2, 6, -1, 4, -3]$, list all states and return an optimal interval. How does allowing an empty interval change all-negative inputs?

A02-E08 Recursion trees and fixed-constant induction

Let $T(1) = 1$ and $T(n) = 3T(n/3) + n$ for powers of three. Find the exact solution by level costs, then verify it by fixed-constant induction. Why does saying that $cn + n$ is still $O(n)$ fail to prove a linear bound?

A02-E09 A logarithm-squared boundary

Consider $T(n) = 9T(n/3) + n^2(\lg n)^2$ with positive constant costs on a fixed base interval. Explain why none of the three classical master-theorem cases applies directly. Derive a tight bound by level sums, first on $n = 3^L$, then justify why floor/ceiling rounding preserves the order.

A02-E10 Unequal splits and perturbations

Analyze $T(n) = T(n/3) + T(2n/3) + n$ with positive bounded base costs. State sufficient hypotheses for the cited Akra–Bazzi theorem, prove existence and uniqueness of the characteristic exponent, and evaluate the integral. Then analyze $T(n) = T(n/2 + n/4) + T(n/2) + n$: why are two one-half ratios invalid?

A02-E11 Seven products and factor order

Using the chapter's definitions of $M_1, \ldots, M_7$, compute every product and $C = AB$ for $A = \begin{bmatrix} 2 & -1 \\ 3 & 4 \end{bmatrix}$ and $B = \begin{bmatrix} 5 & 2 \\ -2 & 1 \end{bmatrix}$. Prove the C_{22} reconstruction identity for general matrix blocks. Why may de not be replaced by ed?

A02-E12 Maximum-subarray merging without scanning

For each nonempty interval, return its total sum, best nonempty prefix sum, best nonempty suffix sum, and best subarray sum. Give all four formulas for merging adjacent intervals, prove them, and analyze total time. Why does this not contradict the CROSS version's $\Theta(n \lg n)$ bound?

A02-E13 Strengthening induction to pay lower-order costs

Let $T(1) = 1, T(n) = 5T(n/2) + n^2$ for powers of two, and $p = \lg 5$. Why does the guess $T(n) \le cn^p$ fail to close induction? Strengthen it to $cn^p - dn^2$, choose fixed c, d, and prove a tight bound.

A02-E14 Rounding and base cases

Is $T(n) = T(\lceil 5n/6 \rceil) + 1$ well-defined for every positive integer if only $T(1) = 1$ is specified? Give a counterexample, a suitable fixed base interval, and a tight bound for the repair. Prove that accumulated rounding error remains bounded by a constant.

A02-E15 Regularity cannot be omitted

On $n = 2^L$, let $f(n) = n^2$ when L is even and $f(n) = n^3$ when L is odd. Define $T(1) = 1$, $T(n) = 2T(n/2) + f(n)$. Prove that $f(n) = \Omega(n^2)$ but $T(n) \ne \Theta(f(n))$. Identify the failed master-theorem hypothesis.

A02-E16 Finer boundaries than polynomial separation

Find tight bounds for $T(n) = 2T(n/2) + n/\lg n$ and $T(n) = 2T(n/2) + n/(\lg n)^2$, on powers of two with fixed positive base costs for $n \le 2$. Write the leaf-end level sums rather than guessing a master-theorem case.

A02-E17 Parameter copying changes time

Binary search starts with an array of original length N. Compare index/view arguments, copying the current length-n interval on every call, and copying the original N records on every call. Give each worst-case recurrence and solution, with its cost rationale. Also explain the cost of an immediate match at the root.

A02-E18 Restore the original variable

On $n = 2^{2^k}$ for $k \geq 0$, define $T(2) = 1$ and $T(n) = 3T(\sqrt{n}) + \lg n$. Find a tight bound, explicitly changing variables, identifying the base, and restoring the original variable. Which variable is confused by a proposed $\Theta(n^{\lg 3})$ answer?

A02-E19 Find the first duplicate

For $A = [1, 2, 2, 2, 5, 8]$, which index does the main binary-search procedure return for 2? Modify it to return the first match. Give a trace, invariant, and termination argument; handle empty input and missing keys.

A02-E20 Exact multiplication counts for powering

Trace every returned value and multiplication count for 2^{13} using the main POWER bases $n = 0, 1$. Prove the formula $\lfloor \lg n \rfloor + \text{popcount}(n) - 1$ for $n \geq 1$. If the identical subcall is evaluated twice, compare call counts, multiplication counts, and peak stack space at $n = 8$.

A02-E21 Three costs in Fibonacci computation

Count base calls in naive Fibonacci recursion. Prove $C_n = 2F_{n+1} - 1$ and compute C_6. Compare naive recursion, sequential addition, and matrix powering in arithmetic operations and stack/value storage. Why do a constant number of retained values not mean a constant number of bits?

A02-E22 Leaf costs in a contracting tree

Use the contracting recurrence from the text, with $T(n) = 1$ for $n \leq 7$. Expand all internal work and leaf costs at $n = 16$, then prove $\Theta(n)$ generally. If local work becomes n^2, what replaces the per-level ratio?

A02-E23 Complete levels do not extend to the deepest leaf

Use unit-cost bases at $n \leq 2$ and split larger sizes into $\lfloor n/3 \rfloor$ and the remainder, with local cost n. At $n = 18$, list all levels, internal-size totals, and leaf costs; compute exact $T(18)$. Why does multiplying maximum depth by 18 construct only an upper bound?

A02-E24 A recurrence inequality need not give a tight bound

Assume only nonnegative cost, $T(1) = 1$, and $T(n) \leq 2T(n/2) + n$ on powers of two. Does this imply $\Theta(n \lg n)$? Give two functions satisfying the assumptions, prove the justified one-sided bound, and explain what additional information would support the matching lower bound.

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing

https://github.com/xinhi6716/computer-science-self-study